Lecture 7: Generative Learning Algorithms

Tao LIN

SoE, Westlake University

October 27, 2026



1. Review of Last Week
   - Exponential Families
   - Generalized Linear Models

2. Generative Learning Algorithms
   - Discriminative vs. Generative Learning Algorithms
   - Gaussian Discriminant Analysis (GDA)
   - Linear Discriminant Analysis (LDA)
   - LDA and Logistic Regression
   - MLE for GDA
   - Naïve Bayes

Reading materials & Reference

Reading materials:

- Chapter 4, Stanford CS 229 Lecture Notes, `https://cs229.stanford.edu/notes2022fall/main_notes.pdf`
- Chapter 9, Probabilistic Machine Learning: an introduction.
- Learning From Data Lecture 4: Generative Learning Algorithms, `http://yangli-feasibility.com/`

Table of Contents

1. Review of Last Week
   - Exponential Families
   - Generalized Linear Models
2. Generative Learning Algorithms

Table of Contents

1. Review of Last Week
   - Exponential Families
   - Generalized Linear Models
2. Generative Learning Algorithms
   - Discriminative vs. Generative Learning Algorithms
   - Gaussian Discriminant Analysis (GDA)
   - Linear Discriminant Analysis (LDA)
   - LDA and Logistic Regression
   - MLE for GDA
   - Naïve Bayes

Logistic Regression and Exponential Families

We start with the Bernoulli model from last lecture. Let $Y \in \{0, 1\}$ and $z = \mathbf{x}^\top \mathbf{w} + b$.

$$\mu(\mathbf{x}) := \mathbb{E}[Y \mid \mathbf{x}] = \Pr(Y = 1 \mid \mathbf{x}) = \sigma(z),$$

Logistic Regression and Exponential Families

We start with the Bernoulli model from last lecture. Let $Y \in \{0, 1\}$ and $z = \mathbf{x}^\top \mathbf{w} + b$.

$$\mu(\mathbf{x}) := \mathbb{E}[Y \mid \mathbf{x}] = \Pr(Y = 1 \mid \mathbf{x}) = \sigma(z),$$
$$p(y \mid \mathbf{x}; \mathbf{w}, b) = \mu^y (1 - \mu)^{1-y} = \exp\left[\eta y - \log(1 + e^\eta)\right].$$

Logistic Regression and Exponential Families

We start with the Bernoulli model from last lecture. Let $Y \in \{0, 1\}$ and $z = \mathbf{x}^\top \mathbf{w} + b$.

$$\mu(\mathbf{x}) := \mathbb{E}[Y \mid \mathbf{x}] = \Pr(Y = 1 \mid \mathbf{x}) = \sigma(z),$$
$$p(y \mid \mathbf{x}; \mathbf{w}, b) = \mu^y (1 - \mu)^{1-y} = \exp\left[\eta y - \log(1 + e^\eta)\right].$$

Here the natural parameter is the log-odds:

$$\eta = \log \frac{\mu}{1 - \mu} = z, \qquad \mu = \sigma(\eta).$$

Logistic Regression and Exponential Families

We start with the Bernoulli model from last lecture. Let $Y \in \{0, 1\}$ and $z = \mathbf{x}^\top \mathbf{w} + b$.

$$\mu(\mathbf{x}) := \mathbb{E}[Y \mid \mathbf{x}] = \Pr(Y = 1 \mid \mathbf{x}) = \sigma(z),$$
$$p(y \mid \mathbf{x}; \mathbf{w}, b) = \mu^y (1 - \mu)^{1-y} = \exp\left[\eta y - \log(1 + e^\eta)\right].$$

Here the natural parameter is the log-odds:

$$\eta = \log \frac{\mu}{1 - \mu} = z, \qquad \mu = \sigma(\eta).$$

Same Bernoulli distribution, two parameterizations:
μ is the probability/mean; η is the log-odds. The canonical link is $g(\mu) = \eta = z$.

Logistic Regression and Exponential Families

We start with the Bernoulli model from last lecture. Let $Y \in \{0, 1\}$ and $z = \mathbf{x}^\top \mathbf{w} + b$.

$$\mu(\mathbf{x}) := \mathbb{E}[Y \mid \mathbf{x}] = \Pr(Y = 1 \mid \mathbf{x}) = \sigma(z),$$
$$p(y \mid \mathbf{x}; \mathbf{w}, b) = \mu^y (1 - \mu)^{1-y} = \exp\left[\eta y - \log(1 + e^\eta)\right].$$

Here the natural parameter is the log-odds:

$$\eta = \log \frac{\mu}{1 - \mu} = z, \qquad \mu = \sigma(\eta).$$

Same Bernoulli distribution, two parameterizations:
μ is the probability/mean; η is the log-odds. The canonical link is $g(\mu) = \eta = z$.

Next question: which parts of this probability formula do we keep fixed, and which part do we learn?

Exponential family — four pieces of one distribution

Use the same scalar template as last lecture:

$$p(y;\eta) = \underbrace{h(y)}_{\text{base measure}} \exp\left[\underbrace{\eta\phi(y)}_{\text{parameter–data interaction}} - \underbrace{A(\eta)}_{\text{log-normalizer}}\right].$$

Exponential family — four pieces of one distribution

Use the same scalar template as last lecture:

$$p(y;\eta) = \underbrace{h(y)}_{\text{base measure}} \exp\left[\underbrace{\eta\phi(y)}_{\text{parameter–data interaction}} - \underbrace{A(\eta)}_{\text{log-normalizer}}\right].$$

- Fix the support, h, and ϕ; vary the natural parameter η.

Exponential family — four pieces of one distribution

Use the same scalar template as last lecture:

$$p(y;\eta) = \underbrace{h(y)}_{\text{base measure}} \exp\left[\underbrace{\eta\phi(y)}_{\text{parameter–data interaction}} - \underbrace{A(\eta)}_{\text{log-normalizer}}\right].$$

- Fix the support, h, and ϕ; vary the natural parameter η.
- For independent observations sharing η, the data enter the likelihood through $\sum_n \phi(y_n)$ (with N fixed).

Exponential family — four pieces of one distribution

Use the same scalar template as last lecture:

$$p(y;\eta) = \underbrace{h(y)}_{\text{base measure}} \exp\left[\underbrace{\eta\phi(y)}_{\text{parameter–data interaction}} - \underbrace{A(\eta)}_{\text{log-normalizer}}\right].$$

- Fix the support, h, and ϕ; vary the natural parameter η.
- For independent observations sharing η, the data enter the likelihood through $\sum_n \phi(y_n)$ (with N fixed).
- The log-partition function makes total probability one:

$$Z(\eta) = \int h(y) e^{\eta\phi(y)}\, dy, \qquad A(\eta) = \log Z(\eta).$$

Exponential family — four pieces of one distribution

Use the same scalar template as last lecture:

$$p(y;\eta) = \underbrace{h(y)}_{\text{base measure}} \exp\left[\underbrace{\eta\phi(y)}_{\text{parameter–data interaction}} - \underbrace{A(\eta)}_{\text{log-normalizer}}\right].$$

- Fix the support, h, and ϕ; vary the natural parameter η.
- For independent observations sharing η, the data enter the likelihood through $\sum_n \phi(y_n)$ (with N fixed).
- The log-partition function makes total probability one:

$$Z(\eta) = \int h(y) e^{\eta\phi(y)}\, dy, \qquad A(\eta) = \log Z(\eta).$$

For discrete outputs, sum over the support. Bernoulli has $Z(\eta) = 1 + e^{\eta}$.

Exponential family — four pieces of one distribution

Use the same scalar template as last lecture:

$$p(y;\eta) = \underbrace{h(y)}_{\text{base measure}} \exp\left[\underbrace{\eta\phi(y)}_{\text{parameter–data interaction}} - \underbrace{A(\eta)}_{\text{log-normalizer}}\right].$$

- Fix the support, h, and ϕ; vary the natural parameter η.
- For independent observations sharing η, the data enter the likelihood through $\sum_n \phi(y_n)$ (with N fixed).
- The log-partition function makes total probability one:

$$Z(\eta) = \int h(y) e^{\eta\phi(y)}\, dy, \qquad A(\eta) = \log Z(\eta).$$

For discrete outputs, sum over the support. Bernoulli has $Z(\eta) = 1 + e^{\eta}$.

Vector extension: replace $\eta\phi(y)$ by $\boldsymbol{\eta}^\top \boldsymbol{\phi}(y)$; h and the normalization rule have the same roles.

Bernoulli: where do the two weights come from?

Same Bernoulli distribution, written with the natural parameter η:

$$p(y;\eta) = \exp[\eta y - A(\eta)] = \frac{e^{\eta y}}{Z(\eta)}, \qquad y \in \{0,1\}.$$

Bernoulli: where do the two weights come from?

Same Bernoulli distribution, written with the natural parameter η:

$$p(y;\eta) = \exp[\eta y - A(\eta)] = \frac{e^{\eta y}}{Z(\eta)}, \qquad y \in \{0,1\}.$$

Substitute each possible outcome:

Outcome $y = 0$

Unnormalized weight: $e^{\eta \cdot 0} = 1$.

$$p(0;\eta) = \frac{1}{Z(\eta)}.$$

Outcome $y = 1$

Unnormalized weight: $e^{\eta \cdot 1} = e^{\eta}$.

$$p(1;\eta) = \frac{e^{\eta}}{Z(\eta)}.$$

Bernoulli: where do the two weights come from?

Same Bernoulli distribution, written with the natural parameter η:

$$p(y;\eta) = \exp[\eta y - A(\eta)] = \frac{e^{\eta y}}{Z(\eta)}, \qquad y \in \{0,1\}.$$

Substitute each possible outcome:

Outcome $y = 0$

Unnormalized weight: $e^{\eta \cdot 0} = 1$.

$$p(0;\eta) = \frac{1}{Z(\eta)}.$$

Outcome $y = 1$

Unnormalized weight: $e^{\eta \cdot 1} = e^{\eta}$.

$$p(1;\eta) = \frac{e^{\eta}}{Z(\eta)}.$$

$$p(0;\eta) + p(1;\eta) = \frac{1 + e^{\eta}}{Z(\eta)} = 1 \quad \Longrightarrow \quad Z(\eta) = 1 + e^{\eta}.$$

The rule: sum the weights to get Z, then divide each weight by Z.

Why does the first derivative give the mean?

Recall Bernoulli: $Z(\eta) = 1 + e^{\eta}$ and $A(\eta) = \log Z(\eta)$.

Two possible outcomes

y	Weight	Probability
0	1	$1/Z$
1	e^{η}	e^{η}/Z

Divide each weight by Z.

Differentiate the log total

Why does the first derivative give the mean?

Recall Bernoulli: $Z(\eta) = 1 + e^{\eta}$ and $A(\eta) = \log Z(\eta)$.

Two possible outcomes

y	Weight	Probability
0	1	$1/Z$
1	e^{η}	e^{η}/Z

Divide each weight by Z.

Differentiate the log total

$$A'(\eta) = \frac{Z'(\eta)}{Z(\eta)} = \frac{0 \cdot 1 + 1 \cdot e^{\eta}}{Z(\eta)}.$$

Differentiating $e^{\eta y}$ brings down the outcome y.

Why does the first derivative give the mean?

Recall Bernoulli: $Z(\eta) = 1 + e^{\eta}$ and $A(\eta) = \log Z(\eta)$.

Two possible outcomes

y	Weight	Probability
0	1	$1/Z$
1	e^{η}	e^{η}/Z

Divide each weight by Z.

Differentiate the log total

$$A'(\eta) = \frac{Z'(\eta)}{Z(\eta)} = \frac{0 \cdot 1 + 1 \cdot e^{\eta}}{Z(\eta)}.$$

Differentiating $e^{\eta y}$ brings down the outcome y.

Read the normalized weights

$$A'(\eta) = 0 \cdot p(0;\eta) + 1 \cdot p(1;\eta).$$

$$= \mathbb{E}_{\eta}[Y] = \mu.$$

Why the mean? The derivative produces outcome $\times$ weight; division by Z turns this into outcome $\times$ probability.

Why does the second derivative give the variance?

From the previous slide: $A' = Z'/Z = \mu$. Differentiate this quotient once more.

Differentiate the weights again

$Z''(\eta) = 0^2 \cdot 1 + 1^2 \cdot e^{\eta}$.

Differentiate the normalized mean

$$A''(\eta) = \frac{Z''}{Z} - \left(\frac{Z'}{Z}\right)^2.$$

Why does the second derivative give the variance?

From the previous slide: $A' = Z'/Z = \mu$. Differentiate this quotient once more.

Differentiate the weights again

$$Z''(\eta) = 0^2 \cdot 1 + 1^2 \cdot e^{\eta}.$$

Divide by the same total weight:

$$\frac{Z''}{Z} = 0^2 p(0;\eta) + 1^2 p(1;\eta).$$

$$= \mathbb{E}_{\eta}[Y^2].$$

Differentiate the normalized mean

$$A''(\eta) = \frac{Z''}{Z} - \left(\frac{Z'}{Z}\right)^2.$$

$$= \mathbb{E}_{\eta}[Y^2] - \mu^2.$$

$$= \mathrm{Var}_{\eta}(Y).$$

Why does the second derivative give the variance?

From the previous slide: $A' = Z'/Z = \mu$. Differentiate this quotient once more.

Differentiate the weights again

$$Z''(\eta) = 0^2 \cdot 1 + 1^2 \cdot e^{\eta}.$$

Divide by the same total weight:

$$\frac{Z''}{Z} = 0^2 p(0;\eta) + 1^2 p(1;\eta).$$

$$= \mathbb{E}_{\eta}[Y^2].$$

Differentiate the normalized mean

$$A''(\eta) = \frac{Z''}{Z} - \left(\frac{Z'}{Z}\right)^2.$$

$$= \mathbb{E}_{\eta}[Y^2] - \mu^2.$$

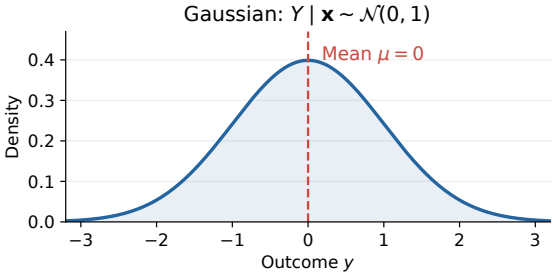
$$= \mathrm{Var}_{\eta}(Y).$$

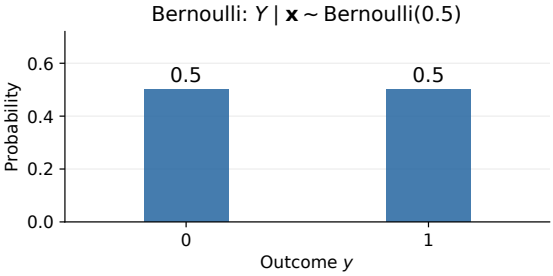
Bernoulli check: $Y^2 = Y \;\Rightarrow\; A''(\eta) = \mu - \mu^2 = \mu(1-\mu)$.

Next question: how do we make the mean μ depend on an input $\mathbf{x}$?

One score, two different predictions

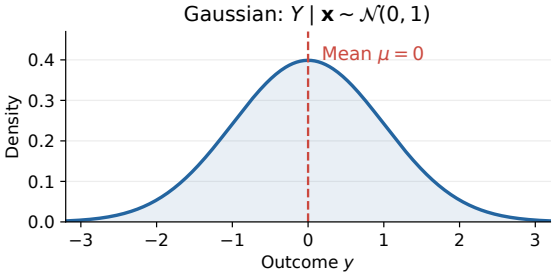
If the score is $z = 0$, must the predicted mean also be zero?

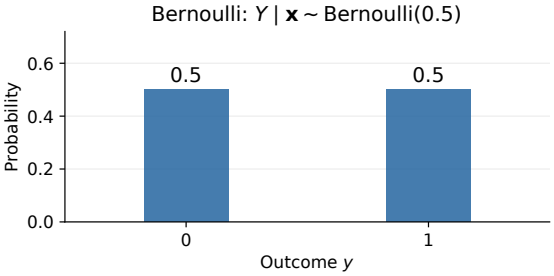




One score, two different predictions

If the score is $z = 0$, must the predicted mean also be zero?





The family matters: Gaussian gives $\mu = z = 0$; Bernoulli gives $\mu = \sigma(z) = 0.5$.

Next: connect the real-valued score to the mean allowed by each family.

Generalized Linear Models: family, score, and link

Predict the conditional mean $\mu(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{x}]$.

Gaussian: real-valued response

$Y \mid \mathbf{x} \sim \mathcal{N}(\mu(\mathbf{x}), 1)$.

Bernoulli: binary response

$Y \mid \mathbf{x} \sim \text{Bernoulli}(\mu(\mathbf{x}))$.

Question: how can a real-valued score produce a valid mean for each family?

Generalized Linear Models: family, score, and link

Predict the conditional mean $\mu(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{x}]$.

Gaussian: real-valued response

$Y \mid \mathbf{x} \sim \mathcal{N}(\mu(\mathbf{x}), 1)$.

Mean range $\mu \in \mathbb{R}$.

Bernoulli: binary response

$Y \mid \mathbf{x} \sim \text{Bernoulli}(\mu(\mathbf{x}))$.

Mean range $\mu \in (0, 1)$ (finite score).

Question: how can a real-valued score produce a valid mean for each family?

Generalized Linear Models: family, score, and link

Predict the conditional mean $\mu(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{x}]$.

Shared score $z = \mathbf{x}^\top \mathbf{w} + b \in \mathbb{R}$. **Link** $g(\mu) = z$.

Gaussian: real-valued response

$Y \mid \mathbf{x} \sim \mathcal{N}(\mu(\mathbf{x}), 1)$.

Mean range $\mu \in \mathbb{R}$.

Bernoulli: binary response

$Y \mid \mathbf{x} \sim \text{Bernoulli}(\mu(\mathbf{x}))$.

Mean range $\mu \in (0, 1)$ (finite score).

Two directions: $g(\mu) = z$ and $\mu = g^{-1}(z)$.

Generalized Linear Models: family, score, and link

Predict the conditional mean $\mu(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{x}]$.

Shared score $z = \mathbf{x}^\top \mathbf{w} + b \in \mathbb{R}$. **Link** $g(\mu) = z$.

Gaussian: real-valued response

$Y \mid \mathbf{x} \sim \mathcal{N}(\mu(\mathbf{x}), 1)$.

Mean range $\mu \in \mathbb{R}$.

Link $g(\mu) = \mu = z$.

Bernoulli: binary response

$Y \mid \mathbf{x} \sim \text{Bernoulli}(\mu(\mathbf{x}))$.

Mean range $\mu \in (0, 1)$ (finite score).

Link $g(\mu) = \log \dfrac{\mu}{1-\mu} = z$.

Two directions: $g(\mu) = z$ and $\mu = g^{-1}(z)$.

Generalized Linear Models: family, score, and link

Predict the conditional mean $\mu(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{x}]$.

Shared score $z = \mathbf{x}^\top \mathbf{w} + b \in \mathbb{R}$. **Link** $g(\mu) = z$.

Gaussian: real-valued response

$Y \mid \mathbf{x} \sim \mathcal{N}(\mu(\mathbf{x}), 1)$.

Mean range $\mu \in \mathbb{R}$.

Link $g(\mu) = \mu = z$.

Mean $\mu = z$.

Bernoulli: binary response

$Y \mid \mathbf{x} \sim \text{Bernoulli}(\mu(\mathbf{x}))$.

Mean range $\mu \in (0, 1)$ (finite score).

Link $g(\mu) = \log \dfrac{\mu}{1-\mu} = z$.

Mean $\mu = \sigma(z)$.

Two directions: $g(\mu) = z$ and $\mu = g^{-1}(z)$.

Generalized Linear Models: family, score, and link

Predict the conditional mean $\mu(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{x}]$.

Shared score $z = \mathbf{x}^\top \mathbf{w} + b \in \mathbb{R}$. **Link** $g(\mu) = z$.

Gaussian: real-valued response

$Y \mid \mathbf{x} \sim \mathcal{N}(\mu(\mathbf{x}), 1)$.

Mean range $\mu \in \mathbb{R}$.

Link $g(\mu) = \mu = z$.

Mean $\mu = z$.

Bernoulli: binary response

$Y \mid \mathbf{x} \sim \text{Bernoulli}(\mu(\mathbf{x}))$.

Mean range $\mu \in (0, 1)$ (finite score).

Link $g(\mu) = \log \dfrac{\mu}{1-\mu} = z$.

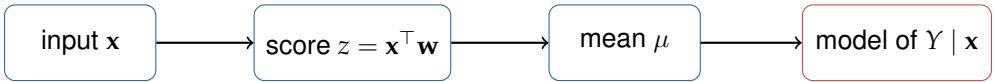
Mean $\mu = \sigma(z)$.

Linear: $g(\mu(\mathbf{x})) = \mathbf{x}^\top \mathbf{w} + b$.

Generalized: the family and link adapt to the response; μ can be nonlinear in z.

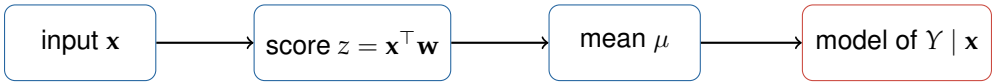
Constructing GLMs: connect the two mappings

Use $\phi(y) = y$; a constant feature absorbs the intercept: $z = \mathbf{x}^\top \mathbf{w}$.



Constructing GLMs: connect the two mappings

Use $\phi(y) = y$; a constant feature absorbs the intercept: $z = \mathbf{x}^\top \mathbf{w}$.



Distribution family

From natural parameter to mean:

$\eta \longrightarrow \mu = A'(\eta).$

Chosen mean model

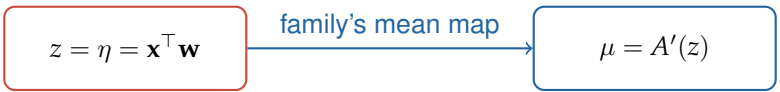
From linear score to mean:

$z \longrightarrow \mu = g^{-1}(z).$

Question: when can we use the score z as the natural parameter η?

Canonical link: let the score be the natural parameter

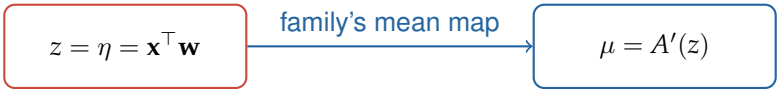
The canonical link is $g(\mu) = \eta$. Since our mean model has $g(\mu) = z$, we get $\eta = z$.



Bernoulli: $\mu = \sigma(z)$. Gaussian, variance 1: $\mu = z$.

Canonical link: let the score be the natural parameter

The canonical link is $g(\mu) = \eta$. Since our mean model has $g(\mu) = z$, we get $\eta = z$.



Bernoulli: $\mu = \sigma(z)$. Gaussian, variance 1: $\mu = z$.

The conditional model is now fixed:

$$p(y \mid \mathbf{x}; \mathbf{w}) = h(y) \exp[zy - A(z)].$$

Next: learn $\mathbf{w}$ by making the observed outputs likely.

Negative log-likelihood: one observed output

Canonical scalar GLM: $z_n = \mathbf{x}_n^\top \mathbf{w}$, $\eta_n = z_n$, and $\phi(y) = y$.

Probability assigned to the observation

$$p_n = p(y_n \mid \mathbf{x}_n; \mathbf{w}) = h(y_n) \exp[z_n y_n - A(z_n)].$$



Negative log-likelihood: one observed output

Canonical scalar GLM: $z_n = \mathbf{x}_n^\top \mathbf{w}$, $\eta_n = z_n$, and $\phi(y) = y$.

Probability assigned to the observation

$$p_n = p(y_n \mid \mathbf{x}_n; \mathbf{w}) = h(y_n) \exp[z_n y_n - A(z_n)].$$

take $-\log$

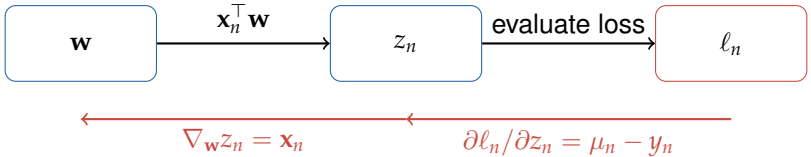
Loss for this observation

$$\ell_n = -\log p_n = A(z_n) - z_n y_n - \log h(y_n).$$

$\log h(y_n)$ is independent of the weights. Higher observed probability means lower loss.

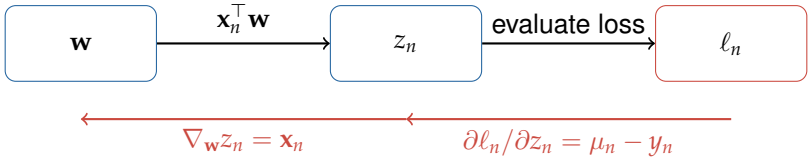
One observation: follow the gradient back to the weights

The model predicts $\mu_n = A'(z_n)$; the observed output is y_n.



One observation: follow the gradient back to the weights

The model predicts $\mu_n = A'(z_n)$; the observed output is y_n.

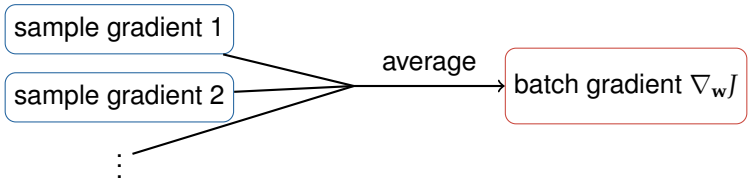


Multiply along the path

$$\nabla_{\mathbf{w}} \ell_n = \underbrace{[A'(z_n) - y_n]}_{\text{predicted mean minus observed output}} \mathbf{x}_n = (\mu_n - y_n)\mathbf{x}_n.$$

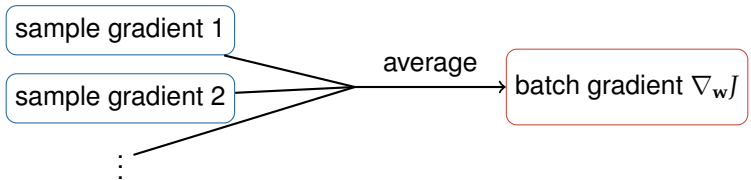
A batch: average the gradients, then update

Independent labels: $L = \prod_n p_n$. Average loss: $J = -\log L/N = \sum_n \ell_n/N$.



A batch: average the gradients, then update

Independent labels: $L = \prod_n p_n$. Average loss: $J = -\log L/N = \sum_n \ell_n/N$.

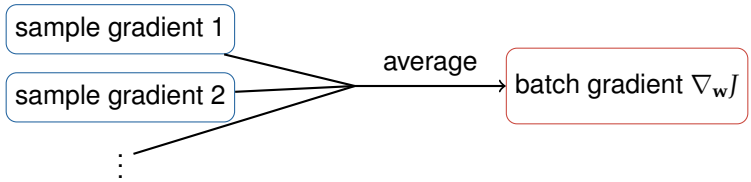


$$\nabla_{\mathbf{w}} J = \frac{1}{N}\sum_n (\mu_n - y_n)\mathbf{x}_n = \frac{1}{N}\mathbf{X}^\top(\boldsymbol{\mu} - \mathbf{y}).$$

$\mathbf{X}$ has rows $\mathbf{x}_n^\top$. For Bernoulli, $\boldsymbol{\mu} = \sigma(\mathbf{X}\mathbf{w})$.

A batch: average the gradients, then update

Independent labels: $L = \prod_n p_n$. Average loss: $J = -\log L/N = \sum_n \ell_n/N$.



$$\nabla_{\mathbf{w}} J = \frac{1}{N}\sum_n (\mu_n - y_n)\mathbf{x}_n = \frac{1}{N}\mathbf{X}^\top(\boldsymbol{\mu} - \mathbf{y}).$$

$\mathbf{X}$ has rows $\mathbf{x}_n^\top$. For Bernoulli, $\boldsymbol{\mu} = \sigma(\mathbf{X}\mathbf{w})$.

Update: $\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla_{\mathbf{w}} J$.

Explicit intercept: $\partial J/\partial b = \sum_n (\mu_n - y_n)/N$.

L2 adds $\lambda\mathbf{w}$ to the weight gradient; the intercept is unpenalized.

From fitting a conditional model to modeling each class

So far: given an input, model its label probability directly:

$$p(1 \mid \mathbf{x}; \mathbf{w}) = \sigma(\mathbf{x}^\top \mathbf{w}).$$

From fitting a conditional model to modeling each class

So far: given an input, model its label probability directly:

$$p(1 \mid \mathbf{x}; \mathbf{w}) = \sigma(\mathbf{x}^\top \mathbf{w}).$$

Next question: could we instead learn what the inputs look like within each class?

From fitting a conditional model to modeling each class

So far: given an input, model its label probability directly:

$$p(1 \mid \mathbf{x}; \mathbf{w}) = \sigma(\mathbf{x}^\top \mathbf{w}).$$

Next question: could we instead learn what the inputs look like within each class?

$$p(\mathbf{x}, y = c; \theta) = \underbrace{\pi_c}_{\text{how common class } c \text{ is}} \underbrace{p(\mathbf{x} \mid y = c; \theta)}_{\text{inputs within class } c}.$$

Here $\pi_c = p(y = c; \theta)$; θ collects the prior and class-model parameters.

From fitting a conditional model to modeling each class

So far: given an input, model its label probability directly:

$$p(1 \mid \mathbf{x}; \mathbf{w}) = \sigma(\mathbf{x}^\top \mathbf{w}).$$

Next question: could we instead learn what the inputs look like within each class?

$$p(\mathbf{x}, y = c; \theta) = \underbrace{\pi_c}_{\text{how common class } c \text{ is}} \underbrace{p(\mathbf{x} \mid y = c; \theta)}_{\text{inputs within class } c}.$$

Here $\pi_c = p(y = c; \theta)$; θ collects the prior and class-model parameters.

Why *generative*? The fitted joint model describes two sampling steps:
choose a class using $p(y)$, then draw an input using $p(\mathbf{x} \mid y)$.

From fitting a conditional model to modeling each class

So far: given an input, model its label probability directly:

$$p(1 \mid \mathbf{x}; \mathbf{w}) = \sigma(\mathbf{x}^\top \mathbf{w}).$$

Next question: could we instead learn what the inputs look like within each class?

$$p(\mathbf{x}, y = c; \theta) = \underbrace{\pi_c}_{\text{how common class } c \text{ is}} \underbrace{p(\mathbf{x} \mid y = c; \theta)}_{\text{inputs within class } c}.$$

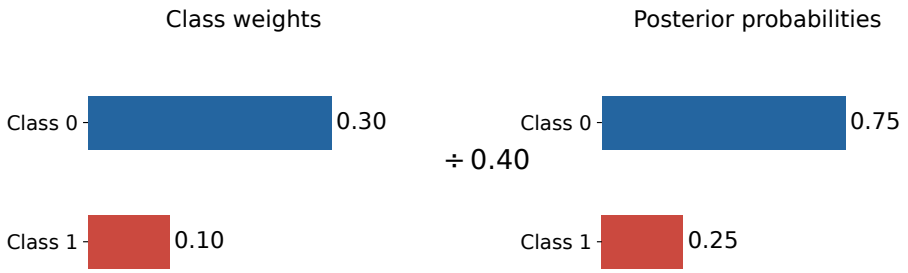
Here $\pi_c = p(y = c; \theta)$; θ collects the prior and class-model parameters.

> Why *generative*? The fitted joint model describes two sampling steps:
> choose a class using $p(y)$, then draw an input using $p(\mathbf{x} \mid y)$.

For classification, reverse the question: we observe $\mathbf{x}$ and infer its class. Next, use Bayes' rule. We keep N observations and d input features.

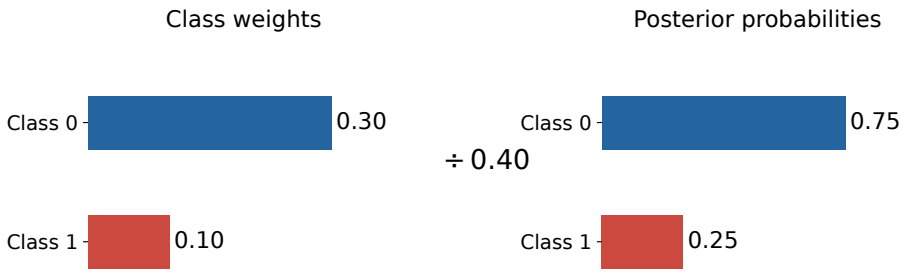
Bayes' rule: rescale the class weights

For one fixed input, each weight already includes its class prior.



Bayes' rule: rescale the class weights

For one fixed input, each weight already includes its class prior.



One shared total: $0.30 + 0.10 = 0.40$. Divide both weights by it.

The probabilities sum to 1; the class ordering stays the same.

Recap: the Bayes classifier

Given a new $\mathbf{x}$, we still want the class with the largest posterior (under 0–1 loss):
$\hat{y}(\mathbf{x}) \in \arg\max_c p(y = c \mid \mathbf{x})$.

Recap: the Bayes classifier

Given a new $\mathbf{x}$, we still want the class with the largest posterior (under 0–1 loss): $\hat{y}(\mathbf{x}) \in \arg\max_c p(y = c \mid \mathbf{x})$. The fitted joint model supplies a weight for each possible class:

$$p(y = c \mid \mathbf{x}; \theta) = \frac{\pi_c \, p(\mathbf{x} \mid y = c; \theta)}{\underbrace{\sum_k \pi_k \, p(\mathbf{x} \mid y = k; \theta)}_{\text{total weight across classes}}}.$$

Recap: the Bayes classifier

Given a new $\mathbf{x}$, we still want the class with the largest posterior (under 0–1 loss): $\hat{y}(\mathbf{x}) \in \arg\max_c p(y = c \mid \mathbf{x})$. The fitted joint model supplies a weight for each possible class:

$$p(y = c \mid \mathbf{x}; \theta) = \frac{\pi_c \, p(\mathbf{x} \mid y = c; \theta)}{\underbrace{\sum_k \pi_k \, p(\mathbf{x} \mid y = k; \theta)}_{\text{total weight across classes}}}.$$

- $p(\mathbf{x} \mid y = c)$: how well does class c's input model match this $\mathbf{x}$? The prior π_c weights that match by how common the class is.

Recap: the Bayes classifier

Given a new $\mathbf{x}$, we still want the class with the largest posterior (under 0–1 loss): $\hat{y}(\mathbf{x}) \in \arg\max_c p(y = c \mid \mathbf{x})$. The fitted joint model supplies a weight for each possible class:

$$p(y = c \mid \mathbf{x}; \theta) = \frac{\pi_c \, p(\mathbf{x} \mid y = c; \theta)}{\underbrace{\sum_k \pi_k \, p(\mathbf{x} \mid y = k; \theta)}_{\text{total weight across classes}}}.$$

- $p(\mathbf{x} \mid y = c)$: how well does class c's input model match this $\mathbf{x}$? The prior π_c weights that match by how common the class is.
- Divide by the total weight across candidate classes: the posterior probabilities sum to one.

Recap: the Bayes classifier

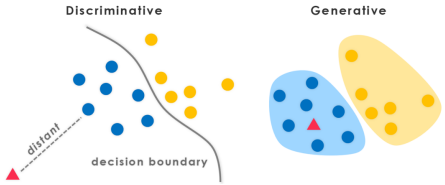
Given a new $\mathbf{x}$, we still want the class with the largest posterior (under 0–1 loss): $\hat{y}(\mathbf{x}) \in \arg\max_c p(y = c \mid \mathbf{x})$. The fitted joint model supplies a weight for each possible class:

$$p(y = c \mid \mathbf{x}; \theta) = \frac{\pi_c \, p(\mathbf{x} \mid y = c; \theta)}{\underbrace{\sum_k \pi_k \, p(\mathbf{x} \mid y = k; \theta)}_{\text{total weight across classes}}}.$$

- $p(\mathbf{x} \mid y = c)$: how well does class c's input model match this $\mathbf{x}$? The prior π_c weights that match by how common the class is.
- Divide by the total weight across candidate classes: the posterior probabilities sum to one.

$p(\mathbf{x} \mid y = c)$ describes inputs within a known class.
$p(y = c \mid \mathbf{x})$ compares classes for a known input. Bayes' rule connects them.

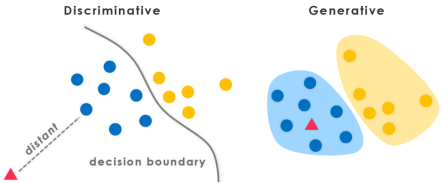
Discriminative vs. Generative Learning Algorithms



Both methods train on the same observed pairs $(\mathbf{x}_n, y_n)$.

Discriminative: fit the conditional **Generative: fit the joint**

Discriminative vs. Generative Learning Algorithms



Both methods train on the same observed pairs $(\mathbf{x}_n, y_n)$.

Discriminative: fit the conditional

$$L(\mathbf{w}, b) = \prod_n p(y_n \mid \mathbf{x}_n; \mathbf{w}, b).$$

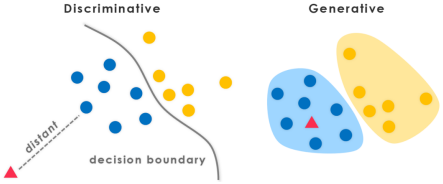
Reward the probability assigned to the observed label at each input.

Generative: fit the joint

$$L(\theta) = \prod_n \pi_{y_n} p(\mathbf{x}_n \mid y_n; \theta).$$

Learn class frequencies and fit the inputs belonging to each observed class.

Discriminative vs. Generative Learning Algorithms



Both methods train on the same observed pairs $(\mathbf{x}_n, y_n)$.

Discriminative: fit the conditional

$$L(\mathbf{w}, b) = \prod_n p(y_n \mid \mathbf{x}_n; \mathbf{w}, b).$$

Reward the probability assigned to the observed label at each input.

Generative: fit the joint

$$L(\theta) = \prod_n \pi_{y_n} p(\mathbf{x}_n \mid y_n; \theta).$$

Learn class frequencies and fit the inputs belonging to each observed class.

At prediction time: both compare $p(y = c \mid \mathbf{x})$. The generative model obtains it from the fitted class models using Bayes' rule.

Generative prediction: what cancels?

For one fixed input with $p(\mathbf{x}; \theta) > 0$,

$$p(y = c \mid \mathbf{x}; \theta) = \frac{\pi_c p(\mathbf{x} \mid y = c; \theta)}{p(\mathbf{x}; \theta)}.$$

Question: which parts change as we compare candidate classes?

Generative prediction: what cancels?

For one fixed input with $p(\mathbf{x}; \theta) > 0$,

$$p(y = c \mid \mathbf{x}; \theta) = \frac{\pi_c p(\mathbf{x} \mid y = c; \theta)}{p(\mathbf{x}; \theta)}.$$

Question: which parts change as we compare candidate classes?

$$\hat{y}(\mathbf{x}) \in \arg\max_c \pi_c p(\mathbf{x} \mid y = c; \theta).$$

The denominator is common to every class, so it cannot change their ordering. The prior does depend on the class and must remain.

Generative prediction: what cancels?

For one fixed input with $p(\mathbf{x}; \theta) > 0$,

$$p(y = c \mid \mathbf{x}; \theta) = \frac{\pi_c p(\mathbf{x} \mid y = c; \theta)}{p(\mathbf{x}; \theta)}.$$

Question: which parts change as we compare candidate classes?

$$\hat{y}(\mathbf{x}) \in \arg\max_c \pi_c p(\mathbf{x} \mid y = c; \theta).$$

The denominator is common to every class, so it cannot change their ordering. The prior does depend on the class and must remain.

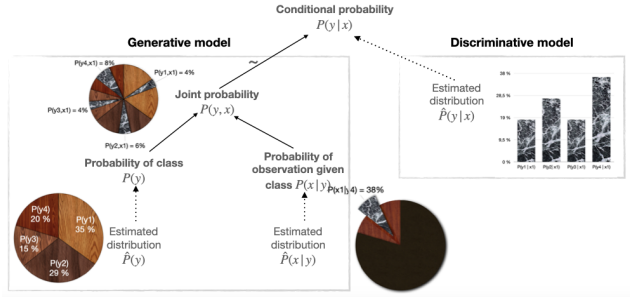
$$\hat{y}(\mathbf{x}) \in \arg\max_c \left[\log p(\mathbf{x} \mid y = c; \theta) + \log \pi_c \right].$$

Interpretation: compare input match plus prior on the log scale. For two classes with equal class-conditional densities at $\mathbf{x}$, the larger prior wins.

Label: compare weights. **Posterior probability:** normalize the weights.

Discriminative vs. Generative: read the same example

Read the two directions: which distributions are fitted before predicting the class?

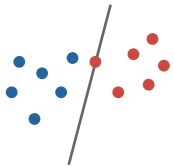


Left: learn an input distribution for each class, then combine it with the prior to obtain $p(y \mid \mathbf{x})$.

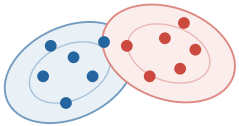
Right: learn $p(y \mid \mathbf{x})$ directly. The output is already the quantity used for classification.

From https://towardsdatascience.com/introduction-to-generative-and-discriminative-models-9c9ef152b9af/

Discriminative vs. Generative: what is assumed?

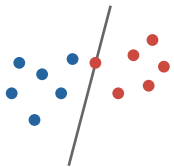


Logistic regression



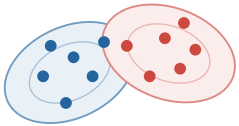
Gaussian class models

Discriminative vs. Generative: what is assumed?



Logistic regression

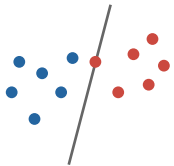
$$\log \frac{p(1 \mid \mathbf{x})}{p(0 \mid \mathbf{x})} = \mathbf{x}^\top \mathbf{w} + b.$$



Gaussian class models

$$\mathbf{x} \mid Y = c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c).$$

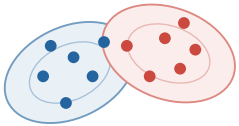
Discriminative vs. Generative: what is assumed?



Logistic regression

$$\log \frac{p(1 \mid \mathbf{x})}{p(0 \mid \mathbf{x})} = \mathbf{x}^\top \mathbf{w} + b.$$

Gaussian class shape?

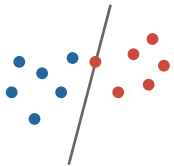


Gaussian class models

$$\mathbf{x} \mid Y = c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c).$$

Gaussian class shape?

Discriminative vs. Generative: what is assumed?

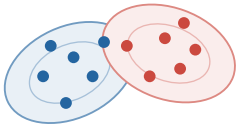


Logistic regression

$$\log \frac{p(1 \mid \mathbf{x})}{p(0 \mid \mathbf{x})} = \mathbf{x}^\top \mathbf{w} + b.$$

Gaussian class shape?

Not required.



Gaussian class models

$$\mathbf{x} \mid Y = c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c).$$

Gaussian class shape?

Required.

Table of Contents

1 Review of Last Week
- Exponential Families
- Generalized Linear Models

2 Generative Learning Algorithms
- Discriminative vs. Generative Learning Algorithms
- Gaussian Discriminant Analysis (GDA)
- Linear Discriminant Analysis (LDA)
- LDA and Logistic Regression
- MLE for GDA
- Naïve Bayes

Gaussian Discriminant Analysis (GDA)

We now need a probability model for the inputs within each class. Recall the Gaussian from last lecture:

$$\text{Last lecture:} \quad Y \mid \mathbf{x} \sim \mathcal{N}(\mathbf{x}^\top \mathbf{w}, 1) \quad \text{(a real-valued response)},$$

Gaussian Discriminant Analysis (GDA)

We now need a probability model for the inputs within each class. Recall the Gaussian from last lecture:

$$\begin{aligned} &\text{Last lecture:} && Y \mid \mathbf{x} \sim \mathcal{N}(\mathbf{x}^\top \mathbf{w}, 1) \quad \text{(a real-valued response)}, \\ &\text{This lecture:} && \mathbf{x} \mid Y = c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \quad \text{(a vector of inputs)}. \end{aligned}$$

Gaussian Discriminant Analysis (GDA)

We now need a probability model for the inputs within each class. Recall the Gaussian from last lecture:

$$\begin{aligned} &\text{Last lecture:} && Y \mid \mathbf{x} \sim \mathcal{N}(\mathbf{x}^\top \mathbf{w}, 1) \quad \text{(a real-valued response)}, \\ &\text{This lecture:} && \mathbf{x} \mid Y = c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \quad \text{(a vector of inputs)}. \end{aligned}$$

- $\boldsymbol{\mu}_c$: mean input within class c; $\boldsymbol{\Sigma}_c$: covariance within that class.

Gaussian Discriminant Analysis (GDA)

We now need a probability model for the inputs within each class. Recall the Gaussian from last lecture:

$$\begin{aligned} &\text{Last lecture:} && Y \mid \mathbf{x} \sim \mathcal{N}(\mathbf{x}^\top \mathbf{w}, 1) \quad \text{(a real-valued response)}, \\ &\text{This lecture:} && \mathbf{x} \mid Y = c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \quad \text{(a vector of inputs)}. \end{aligned}$$

- $\boldsymbol{\mu}_c$: mean input within class c; $\boldsymbol{\Sigma}_c$: covariance within that class.
- π_c: how common class c is. Bayes' rule then gives $p(y = c \mid \mathbf{x}; \theta) \propto \pi_c \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$.

Gaussian Discriminant Analysis (GDA)

We now need a probability model for the inputs within each class. Recall the Gaussian from last lecture:

$$\begin{aligned} &\text{Last lecture:} && Y \mid \mathbf{x} \sim \mathcal{N}(\mathbf{x}^\top \mathbf{w}, 1) \quad \text{(a real-valued response)}, \\ &\text{This lecture:} && \mathbf{x} \mid Y = c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \quad \text{(a vector of inputs)}. \end{aligned}$$

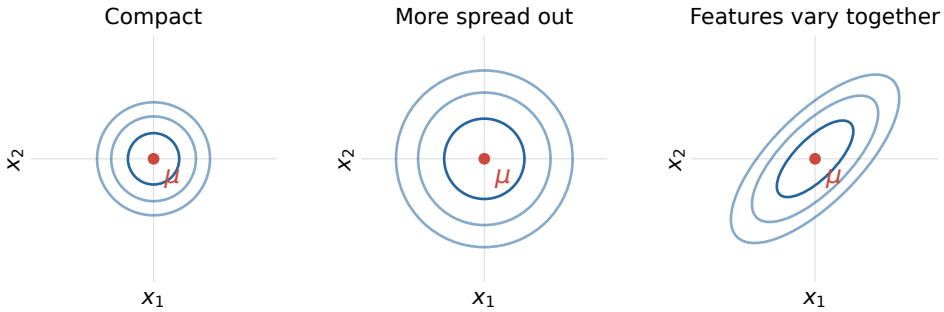
- $\boldsymbol{\mu}_c$: mean input within class c; $\boldsymbol{\Sigma}_c$: covariance within that class.
- π_c: how common class c is. Bayes' rule then gives $p(y = c \mid \mathbf{x}; \theta) \propto \pi_c \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$.

Same Gaussian family; we now model $\mathbf{x} \mid Y = c$ to predict a class label.

Next question: what changes when a Gaussian observation is a vector?

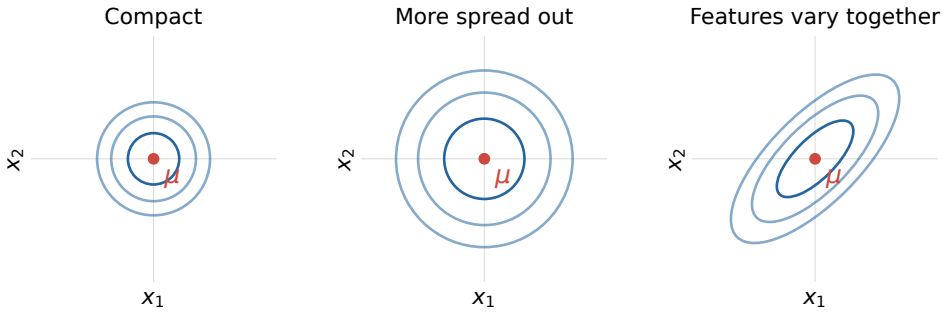
Read a Gaussian as a contour map

Each contour connects inputs with the same density within that panel.



Read a Gaussian as a contour map

Each contour connects inputs with the same density within that panel.



Mean μ: center of the contours. **Covariance Σ:** spread and orientation.

Next: read these two roles in the multivariate Gaussian formula.

The multivariate normal distribution

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{\exp[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})]}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}}.$$

The multivariate normal distribution

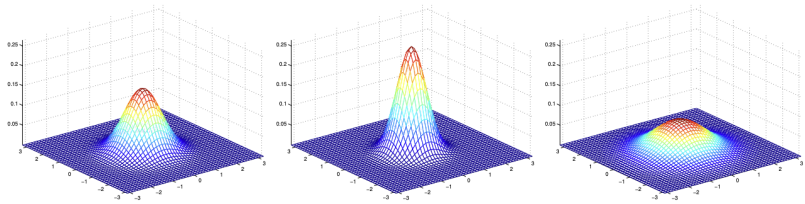
$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{\exp[-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})]}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}}.$$

$\boldsymbol{\mu} \in \mathbb{R}^d$; $\boldsymbol{\Sigma} \in \mathbb{R}^{d \times d}$ is symmetric and **positive definite** for this full-dimensional density.

The multivariate normal distribution

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{\exp[-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})]}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}}.$$

$\boldsymbol{\mu} \in \mathbb{R}^d$; $\boldsymbol{\Sigma} \in \mathbb{R}^{d \times d}$ is symmetric and **positive definite** for this full-dimensional density.

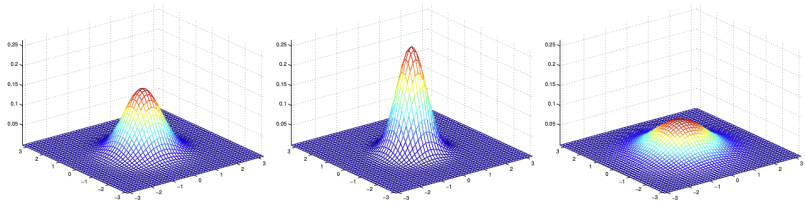


Left: $\boldsymbol{\Sigma} = \mathbf{I}$; middle: $0.6\mathbf{I}$; right: $2\mathbf{I}$. Larger isotropic variance gives a more spread-out density.

The multivariate normal distribution

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{\exp[-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})]}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}}.$$

$\boldsymbol{\mu} \in \mathbb{R}^d$; $\boldsymbol{\Sigma} \in \mathbb{R}^{d \times d}$ is symmetric and **positive definite** for this full-dimensional density.

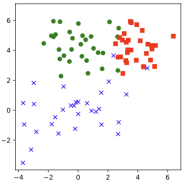


Left: $\boldsymbol{\Sigma} = \mathbf{I}$; middle: $0.6\mathbf{I}$; right: $2\mathbf{I}$. Larger isotropic variance gives a more spread-out density. A covariance may be semidefinite in general; a singular Gaussian has no density of the above form.

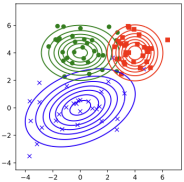
Gaussian Discriminant Analysis (GDA) Model

For C classes, let $\pi_c > 0$, $\sum_c \pi_c = 1$, and

$$Y \sim \text{Categorical}(\pi_1, \ldots, \pi_C), \qquad \mathbf{x} \mid Y = c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c).$$



(a)

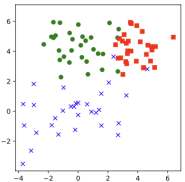


(b)

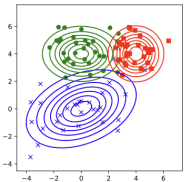
Gaussian Discriminant Analysis (GDA) Model

For C classes, let $\pi_c > 0$, $\sum_c \pi_c = 1$, and

$$Y \sim \text{Categorical}(\pi_1, \ldots, \pi_C), \qquad \mathbf{x} \mid Y = c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c).$$



(a)



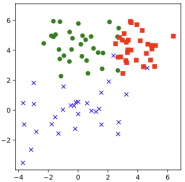
(b)

For binary labels $Y \in \{0, 1\}$, write $Y \sim \text{Bernoulli}(\pi_1)$, with $\pi_0 = 1 - \pi_1$.

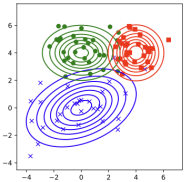
Gaussian Discriminant Analysis (GDA) Model

For C classes, let $\pi_c > 0$, $\sum_c \pi_c = 1$, and

$$Y \sim \text{Categorical}(\pi_1, \ldots, \pi_C), \qquad \mathbf{x} \mid Y = c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c).$$



(a)



(b)

For binary labels $Y \in \{0, 1\}$, write $Y \sim \text{Bernoulli}(\pi_1)$, with $\pi_0 = 1 - \pi_1$.

Two choices: class-specific covariance $\boldsymbol{\Sigma}_c$ (QDA), or shared covariance $\boldsymbol{\Sigma}_c = \boldsymbol{\Sigma}$ (LDA).

GDA Model: from densities to class scores

As in the GLM likelihood, taking a log turns products into sums. For each class, define

$$s_c(\mathbf{x}) := \log \left[\pi_c \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \right]$$

GDA Model: from densities to class scores

As in the GLM likelihood, taking a log turns products into sums. For each class, define

$$
\begin{aligned}
s_c(\mathbf{x}) &:= \log\left[\pi_c \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)\right] \\
&= \log \pi_c - \frac{1}{2}\log|\boldsymbol{\Sigma}_c| - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}_c^{-1}(\mathbf{x}-\boldsymbol{\mu}_c) \\
&\quad - \frac{d}{2}\log(2\pi).
\end{aligned}
$$

GDA Model: from densities to class scores

As in the GLM likelihood, taking a log turns products into sums. For each class, define

$$
\begin{aligned}
s_c(\mathbf{x}) &:= \log\left[\pi_c \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)\right] \\
&= \log \pi_c - \frac{1}{2}\log|\boldsymbol{\Sigma}_c| - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}_c^{-1}(\mathbf{x}-\boldsymbol{\mu}_c) \\
&\quad - \frac{d}{2}\log(2\pi).
\end{aligned}
$$

$$
p(y = c \mid \mathbf{x}; \theta) = \frac{e^{s_c(\mathbf{x})}}{\sum_k e^{s_k(\mathbf{x})}}, \qquad \hat{y}(\mathbf{x}) \in \arg\max_c s_c(\mathbf{x}).
$$

GDA Model: from densities to class scores

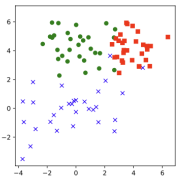
As in the GLM likelihood, taking a log turns products into sums. For each class, define

$$\begin{aligned} s_c(\mathbf{x}) &:= \log\left[\pi_c \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)\right] \\ &= \log \pi_c - \frac{1}{2}\log|\boldsymbol{\Sigma}_c| - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}_c^{-1}(\mathbf{x}-\boldsymbol{\mu}_c) \\ &\quad - \frac{d}{2}\log(2\pi). \end{aligned}$$

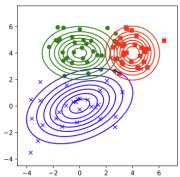
$$p(y=c \mid \mathbf{x};\theta) = \frac{e^{s_c(\mathbf{x})}}{\sum_k e^{s_k(\mathbf{x})}}, \qquad \hat{y}(\mathbf{x}) \in \arg\max_c s_c(\mathbf{x}).$$

What stays the same across classes? The last term is common; the log-determinant term is common when covariances are shared. Next, compare the boundaries with and without sharing.

GDA geometry: separate or shared covariance?



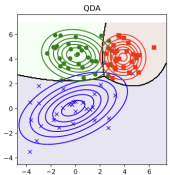
(a)



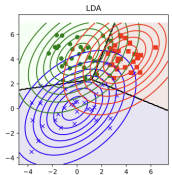
(b)

Class-specific Σ_c: QDA

Pairwise boundaries are generally quadratic.



(a)

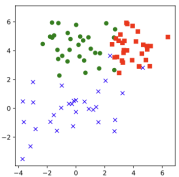


(b)

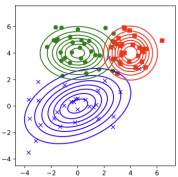
Shared Σ: LDA

Pairwise boundaries are linear.

GDA geometry: separate or shared covariance?

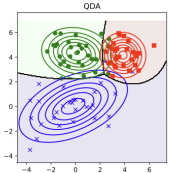


(a)

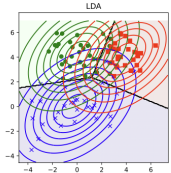


(b)

Class-specific Σ_c: QDA
Pairwise boundaries are generally quadratic.



(a)



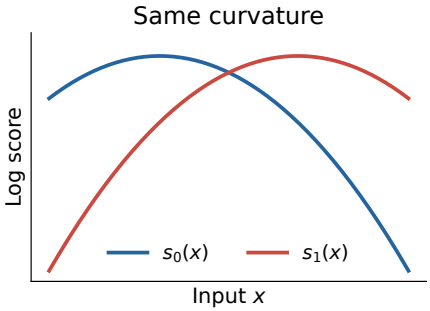
(b)

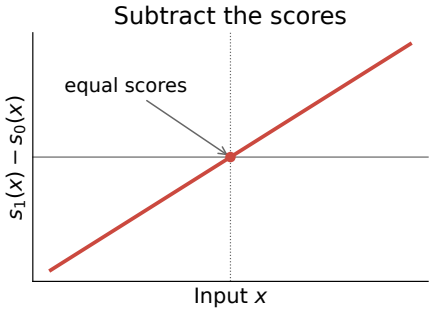
Shared Σ: LDA
Pairwise boundaries are linear.

Next question: which quadratic terms cancel when $\Sigma_c = \Sigma$?

Shared covariance: the curvature cancels

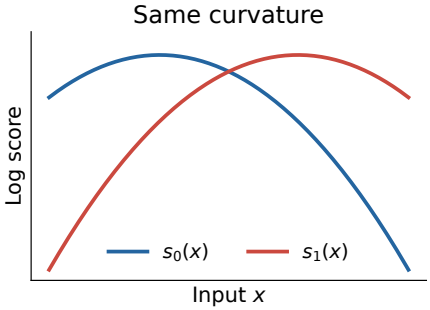
Look at one input dimension, with equal priors and a shared variance.

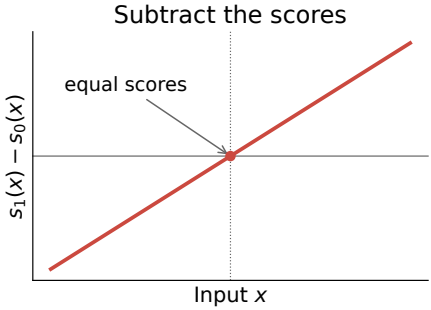




Shared covariance: the curvature cancels

Look at one input dimension, with equal priors and a shared variance.





Two curved log scores can have a **straight-line difference**.

Decision boundary: where their difference is zero. Next, find the cancelling term.

What is LDA?

LDA shares one covariance matrix across all classes.

Why does this make the decision boundary linear?

What is LDA?

LDA shares one covariance matrix across all classes.

Why does this make the decision boundary linear?

$$(\mathbf{x} - \boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}_c) = \mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \mathbf{x} - 2\mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c + \boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c.$$

What is LDA?

LDA shares one covariance matrix across all classes.

Why does this make the decision boundary linear?

$$(\mathbf{x} - \boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}_c) = \mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \mathbf{x} - 2\mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c + \boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c.$$

The first term depends on $\mathbf{x}$, but **not on the class**. It cancels when comparing classes.

Table of Contents

1 Review of Last Week
- Exponential Families
- Generalized Linear Models

2 Generative Learning Algorithms
- Discriminative vs. Generative Learning Algorithms
- Gaussian Discriminant Analysis (GDA)
- Linear Discriminant Analysis (LDA)
- LDA and Logistic Regression
- MLE for GDA
- Naïve Bayes

Linear Discriminant Analysis (LDA)

With $\boldsymbol{\Sigma}_c = \boldsymbol{\Sigma}$, expand the Gaussian log score:

$$s_c(\mathbf{x}) = \log \pi_c - \frac{1}{2}\boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c + \mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c + \kappa(\mathbf{x}),$$

Linear Discriminant Analysis (LDA)

With $\boldsymbol{\Sigma}_c = \boldsymbol{\Sigma}$, expand the Gaussian log score:

$$s_c(\mathbf{x}) = \log \pi_c - \frac{1}{2}\boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c + \mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c + \kappa(\mathbf{x}),$$

$$\gamma_c := \log \pi_c - \frac{1}{2}\boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c, \qquad \boldsymbol{\beta}_c := \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c.$$

Linear Discriminant Analysis (LDA)

With $\Sigma_c = \Sigma$, expand the Gaussian log score:

$$s_c(\mathbf{x}) = \log \pi_c - \frac{1}{2}\boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c + \mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c + \kappa(\mathbf{x}),$$

$$\gamma_c := \log \pi_c - \frac{1}{2}\boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c, \qquad \boldsymbol{\beta}_c := \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c.$$

$$\delta_c(\mathbf{x}) := \gamma_c + \mathbf{x}^\top \boldsymbol{\beta}_c, \qquad \hat{y}(\mathbf{x}) \in \arg\max_c \delta_c(\mathbf{x}).$$

Linear Discriminant Analysis (LDA)

With $\Sigma_c = \Sigma$, expand the Gaussian log score:

$$s_c(\mathbf{x}) = \log \pi_c - \frac{1}{2}\boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c + \mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c + \kappa(\mathbf{x}),$$

$$\gamma_c := \log \pi_c - \frac{1}{2}\boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c, \qquad \boldsymbol{\beta}_c := \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c.$$

$$\delta_c(\mathbf{x}) := \gamma_c + \mathbf{x}^\top \boldsymbol{\beta}_c, \qquad \hat{y}(\mathbf{x}) \in \arg\max_c \delta_c(\mathbf{x}).$$

- $\kappa(\mathbf{x})$ contains all terms shared by the classes.

Linear Discriminant Analysis (LDA)

With $\Sigma_c = \Sigma$, expand the Gaussian log score:

$$s_c(\mathbf{x}) = \log \pi_c - \frac{1}{2}\boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c + \mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c + \kappa(\mathbf{x}),$$

$$\gamma_c := \log \pi_c - \frac{1}{2}\boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c, \qquad \boldsymbol{\beta}_c := \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c.$$

$$\delta_c(\mathbf{x}) := \gamma_c + \mathbf{x}^\top \boldsymbol{\beta}_c, \qquad \hat{y}(\mathbf{x}) \in \arg\max_c \delta_c(\mathbf{x}).$$

- $\kappa(\mathbf{x})$ contains all terms shared by the classes.
- Boundary between c and k: $(\boldsymbol{\beta}_c - \boldsymbol{\beta}_k)^\top \mathbf{x} + (\gamma_c - \gamma_k) = 0$.

Table of Contents

1 Review of Last Week
- Exponential Families
- Generalized Linear Models

2 Generative Learning Algorithms
- Discriminative vs. Generative Learning Algorithms
- Gaussian Discriminant Analysis (GDA)
- Linear Discriminant Analysis (LDA)
- LDA and Logistic Regression
- MLE for GDA
- Naïve Bayes

The connection between LDA and logistic regression

Recall $s_c(\mathbf{x}) = \delta_c(\mathbf{x}) + \kappa(\mathbf{x})$, where $\delta_c = \gamma_c + \mathbf{x}^\top \boldsymbol{\beta}_c$.

$$p(y = c \mid \mathbf{x}; \theta) = \frac{e^{\delta_c(\mathbf{x}) + \kappa(\mathbf{x})}}{\sum_k e^{\delta_k(\mathbf{x}) + \kappa(\mathbf{x})}}$$

The connection between LDA and logistic regression

Recall $s_c(\mathbf{x}) = \delta_c(\mathbf{x}) + \kappa(\mathbf{x})$, where $\delta_c = \gamma_c + \mathbf{x}^\top \boldsymbol{\beta}_c$.

$$
\begin{aligned}
p(y = c \mid \mathbf{x}; \theta) &= \frac{e^{\delta_c(\mathbf{x}) + \kappa(\mathbf{x})}}{\sum_k e^{\delta_k(\mathbf{x}) + \kappa(\mathbf{x})}} \\
&= \frac{e^{\gamma_c + \mathbf{x}^\top \boldsymbol{\beta}_c}}{\sum_k e^{\gamma_k + \mathbf{x}^\top \boldsymbol{\beta}_k}}.
\end{aligned}
$$

The connection between LDA and logistic regression

Recall $s_c(\mathbf{x}) = \delta_c(\mathbf{x}) + \kappa(\mathbf{x})$, where $\delta_c = \gamma_c + \mathbf{x}^\top \boldsymbol{\beta}_c$.

$$p(y = c \mid \mathbf{x}; \theta) = \frac{e^{\delta_c(\mathbf{x}) + \kappa(\mathbf{x})}}{\sum_k e^{\delta_k(\mathbf{x}) + \kappa(\mathbf{x})}}$$
$$= \frac{e^{\gamma_c + \mathbf{x}^\top \boldsymbol{\beta}_c}}{\sum_k e^{\gamma_k + \mathbf{x}^\top \boldsymbol{\beta}_k}}.$$

Shared-covariance Gaussian class models give a **softmax posterior** with linear scores.

The connection between LDA and logistic regression

Recall $s_c(\mathbf{x}) = \delta_c(\mathbf{x}) + \kappa(\mathbf{x})$, where $\delta_c = \gamma_c + \mathbf{x}^\top \boldsymbol{\beta}_c$.

$$p(y = c \mid \mathbf{x}; \theta) = \frac{e^{\delta_c(\mathbf{x}) + \kappa(\mathbf{x})}}{\sum_k e^{\delta_k(\mathbf{x}) + \kappa(\mathbf{x})}} = \frac{e^{\gamma_c + \mathbf{x}^\top \boldsymbol{\beta}_c}}{\sum_k e^{\gamma_k + \mathbf{x}^\top \boldsymbol{\beta}_k}}.$$

Shared-covariance Gaussian class models give a **softmax posterior** with linear scores.

For two classes, softmax reduces to the sigmoid from Lecture 06.

Binary LDA: recover the sigmoid

For two classes, recall the Bernoulli log-odds:

$$\eta(\mathbf{x}) := \log \frac{p(1 \mid \mathbf{x}; \theta)}{p(0 \mid \mathbf{x}; \theta)} = \delta_1(\mathbf{x}) - \delta_0(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} + b,$$

Binary LDA: recover the sigmoid

For two classes, recall the Bernoulli log-odds:

$$\eta(\mathbf{x}) := \log \frac{p(1 \mid \mathbf{x}; \theta)}{p(0 \mid \mathbf{x}; \theta)} = \delta_1(\mathbf{x}) - \delta_0(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} + b,$$

$$\mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0),$$

$$b = -\frac{1}{2}(\boldsymbol{\mu}_1^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_1 - \boldsymbol{\mu}_0^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_0) + \log \frac{\pi_1}{\pi_0}.$$

Binary LDA: recover the sigmoid

For two classes, recall the Bernoulli log-odds:

$$\eta(\mathbf{x}) := \log \frac{p(1 \mid \mathbf{x}; \theta)}{p(0 \mid \mathbf{x}; \theta)} = \delta_1(\mathbf{x}) - \delta_0(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} + b,$$

$$\mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0),$$

$$b = -\frac{1}{2}(\boldsymbol{\mu}_1^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_1 - \boldsymbol{\mu}_0^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_0) + \log \frac{\pi_1}{\pi_0}.$$

The same mean–parameter relation gives

$$\mu(\mathbf{x}) = p(1 \mid \mathbf{x}; \theta) = \sigma(\eta(\mathbf{x})) = \sigma(\mathbf{x}^\top \mathbf{w} + b).$$

Binary LDA: recover the sigmoid

For two classes, recall the Bernoulli log-odds:

$$\eta(\mathbf{x}) := \log \frac{p(1 \mid \mathbf{x}; \theta)}{p(0 \mid \mathbf{x}; \theta)} = \delta_1(\mathbf{x}) - \delta_0(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} + b,$$

$$\mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0),$$

$$b = -\frac{1}{2}(\boldsymbol{\mu}_1^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_1 - \boldsymbol{\mu}_0^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_0) + \log \frac{\pi_1}{\pi_0}.$$

The same mean–parameter relation gives

$$\mu(\mathbf{x}) = p(1 \mid \mathbf{x}; \theta) = \sigma(\eta(\mathbf{x})) = \sigma(\mathbf{x}^\top \mathbf{w} + b).$$

Same Bernoulli model: linear log-odds η, mean $\mu = \sigma(\eta)$.
Here $\mathbf{w}, b$ come from the fitted Gaussian class models.

Binary LDA: recover the sigmoid

For two classes, recall the Bernoulli log-odds:

$$\eta(\mathbf{x}) := \log \frac{p(1 \mid \mathbf{x}; \theta)}{p(0 \mid \mathbf{x}; \theta)} = \delta_1(\mathbf{x}) - \delta_0(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} + b,$$

$$\mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0),$$

$$b = -\frac{1}{2}(\boldsymbol{\mu}_1^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_1 - \boldsymbol{\mu}_0^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_0) + \log \frac{\pi_1}{\pi_0}.$$

The same mean–parameter relation gives

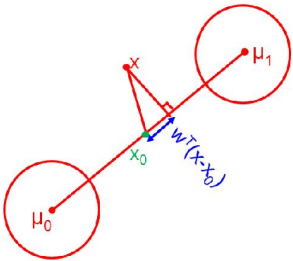
$$\mu(\mathbf{x}) = p(1 \mid \mathbf{x}; \theta) = \sigma(\eta(\mathbf{x})) = \sigma(\mathbf{x}^\top \mathbf{w} + b).$$

Same Bernoulli model: linear log-odds η, mean $\mu = \sigma(\eta)$.
Here $\mathbf{w}, b$ come from the fitted Gaussian class models.

Next question: where is the decision boundary?

Geometry interpretation of LDA

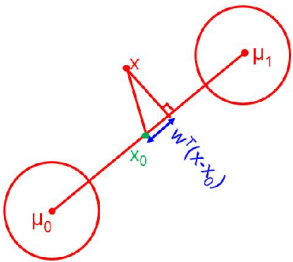
For two classes, choose class 1 when $\mathbf{x}^\top \mathbf{w} + b > 0$ (ties arbitrary).



Geometry interpretation of LDA

For two classes, choose class 1 when $\mathbf{x}^\top \mathbf{w} + b > 0$ (ties arbitrary).

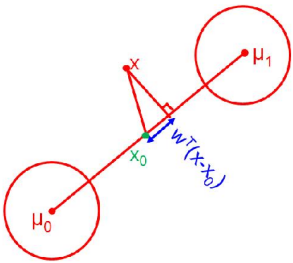
- Equal priors: the boundary passes through $\mathbf{x}_0 = (\boldsymbol{\mu}_0 + \boldsymbol{\mu}_1)/2$.



Geometry interpretation of LDA

For two classes, choose class 1 when $\mathbf{x}^\top \mathbf{w} + b > 0$ (ties arbitrary).

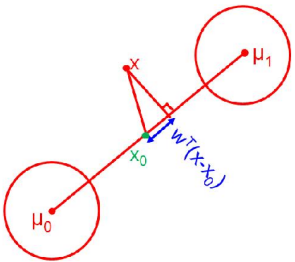
- Equal priors: the boundary passes through $\mathbf{x}_0 = (\boldsymbol{\mu}_0 + \boldsymbol{\mu}_1)/2$.
- If $\boldsymbol{\Sigma} = \sigma^2 \mathbf{I}$, $\mathbf{w} = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)/\sigma^2$ is parallel to the line joining the means.



Geometry interpretation of LDA

For two classes, choose class 1 when $\mathbf{x}^\top \mathbf{w} + b > 0$ (ties arbitrary).

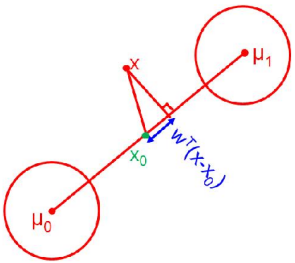
- Equal priors: the boundary passes through $\mathbf{x}_0 = (\boldsymbol{\mu}_0 + \boldsymbol{\mu}_1)/2$.
- If $\boldsymbol{\Sigma} = \sigma^2 \mathbf{I}$, $\mathbf{w} = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)/\sigma^2$ is parallel to the line joining the means.
- With both assumptions, choose the nearest mean in Euclidean distance.



Geometry interpretation of LDA

For two classes, choose class 1 when $\mathbf{x}^\top \mathbf{w} + b > 0$ (ties arbitrary).

- Equal priors: the boundary passes through $\mathbf{x}_0 = (\boldsymbol{\mu}_0 + \boldsymbol{\mu}_1)/2$.
- If $\boldsymbol{\Sigma} = \sigma^2 \mathbf{I}$, $\mathbf{w} = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)/\sigma^2$ is parallel to the line joining the means.
- With both assumptions, choose the nearest mean in Euclidean distance.
- A general shared covariance changes the direction; unequal priors shift the boundary through b.



LDA by hand: how does the prior move the boundary?

One-dimensional example: $\mu_0 = 0$, $\mu_1 = 2$, shared variance $\sigma^2 = 1$.

$$w = 2, \qquad b = -2 + \log(\pi_1/\pi_0),$$
$$p(1 \mid x) = \sigma\left(2x - 2 + \log(\pi_1/\pi_0)\right).$$

LDA by hand: how does the prior move the boundary?

One-dimensional example: $\mu_0 = 0$, $\mu_1 = 2$, shared variance $\sigma^2 = 1$.

$$w = 2, \qquad b = -2 + \log(\pi_1/\pi_0),$$
$$p(1 \mid x) = \sigma\left(2x - 2 + \log(\pi_1/\pi_0)\right).$$

- Equal priors: boundary $x = 1$; at $x = 1$, posterior $= 0.5$.

LDA by hand: how does the prior move the boundary?

One-dimensional example: $\mu_0 = 0$, $\mu_1 = 2$, shared variance $\sigma^2 = 1$.

$$w = 2, \qquad b = -2 + \log(\pi_1/\pi_0),$$
$$p(1 \mid x) = \sigma\left(2x - 2 + \log(\pi_1/\pi_0)\right).$$

- Equal priors: boundary $x = 1$; at $x = 1$, posterior $= 0.5$.
- $\pi_1 = 0.8$, $\pi_0 = 0.2$: boundary $x = 1 - \frac{1}{2}\log 4 \approx 0.307$.

LDA by hand: how does the prior move the boundary?

One-dimensional example: $\mu_0 = 0$, $\mu_1 = 2$, shared variance $\sigma^2 = 1$.

$$w = 2, \qquad b = -2 + \log(\pi_1/\pi_0),$$
$$p(1 \mid x) = \sigma\left(2x - 2 + \log(\pi_1/\pi_0)\right).$$

- Equal priors: boundary $x = 1$; at $x = 1$, posterior $= 0.5$.
- $\pi_1 = 0.8$, $\pi_0 = 0.2$: boundary $x = 1 - \frac{1}{2}\log 4 \approx 0.307$.
- At $x = 1$, the likelihoods are equal, but the posterior is now 0.8.

LDA by hand: how does the prior move the boundary?

One-dimensional example: $\mu_0 = 0$, $\mu_1 = 2$, shared variance $\sigma^2 = 1$.

$$w = 2, \qquad b = -2 + \log(\pi_1/\pi_0),$$
$$p(1 \mid x) = \sigma\left(2x - 2 + \log(\pi_1/\pi_0)\right).$$

- Equal priors: boundary $x = 1$; at $x = 1$, posterior $= 0.5$.
- $\pi_1 = 0.8$, $\pi_0 = 0.2$: boundary $x = 1 - \frac{1}{2}\log 4 \approx 0.307$.
- At $x = 1$, the likelihoods are equal, but the posterior is now 0.8.

Explain: a more common class needs less likelihood evidence to be predicted.

Some remarks: same posterior form, different fitting

We have derived the same sigmoid form. Compare what the two fitting procedures reward:

LDA: fit labeled inputs jointly

$$L(\theta) = \prod_n p(\mathbf{x}_n, y_n; \theta).$$

Fit the class priors and shared-covariance Gaussian class models.

Logistic: fit conditional labels

$$L(\mathbf{w}, b) = \prod_n p(y_n \mid \mathbf{x}_n; \mathbf{w}, b).$$

Fit a sigmoid with linear log-odds directly.

Question: why can the fitted coefficients differ on the same data?

Some remarks: same posterior form, different fitting

We have derived the same sigmoid form. Compare what the two fitting procedures reward:

LDA: fit labeled inputs jointly

$$L(\theta) = \prod_n p(\mathbf{x}_n, y_n; \theta).$$

Fit the class priors and shared-covariance Gaussian class models.

Then derive $\mathbf{w}, b$ from those fitted parameters.

Logistic: fit conditional labels

$$L(\mathbf{w}, b) = \prod_n p(y_n \mid \mathbf{x}_n; \mathbf{w}, b).$$

Fit a sigmoid with linear log-odds directly.

Optimize $\mathbf{w}, b$ using conditional NLL; no model for $p(\mathbf{x})$ is fitted.

Question: why can the fitted coefficients differ on the same data?

Some remarks: same posterior form, different fitting

Expand the joint likelihood using the conditional probability it induces:

$$\begin{aligned}\log L(\theta) &= \sum_n \log p(\mathbf{x}_n, y_n; \theta) \\ &= \underbrace{\sum_n \log p(y_n \mid \mathbf{x}_n; \theta)}_{\text{fit the labels given the inputs}} + \underbrace{\sum_n \log p(\mathbf{x}_n; \theta)}_{\text{also fit the inputs}}.\end{aligned}$$

The same θ controls both terms. Joint fitting must explain the inputs as well as their labels; conditional fitting rewards only the label probabilities.

Some remarks: same posterior form, different fitting

Expand the joint likelihood using the conditional probability it induces:

$$\begin{aligned}\log L(\theta) &= \sum_n \log p(\mathbf{x}_n, y_n; \theta) \\ &= \underbrace{\sum_n \log p(y_n \mid \mathbf{x}_n; \theta)}_{\text{fit the labels given the inputs}} + \underbrace{\sum_n \log p(\mathbf{x}_n; \theta)}_{\text{also fit the inputs}}.\end{aligned}$$

The same θ controls both terms. Joint fitting must explain the inputs as well as their labels; conditional fitting rewards only the label probabilities.

The sigmoid describes how the fitted model predicts.

The likelihood defines how its parameters are learned.

Same posterior family does not imply the same fitted coefficients.

Quick check: why does LDA give a sigmoid?

Which statement is correct for two-class LDA?

A. Any two Gaussian class models always give linear log-odds.

B. Shared covariance cancels the quadratic input terms, leaving linear log-odds.

C. LDA and logistic regression maximize the same likelihood.

Think / **vote:** identify the term that cancels.

Quick check: why does LDA give a sigmoid?

Which statement is correct for two-class LDA?

A. Any two Gaussian class models always give linear log-odds.

B. Shared covariance cancels the quadratic input terms, leaving linear log-odds.

C. LDA and logistic regression maximize the same likelihood.

Answer: B. The shared term $-\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x}/2$ cancels. Different covariances generally leave quadratic terms.

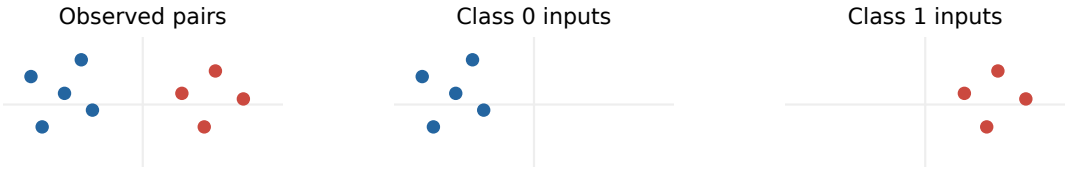
Table of Contents

1 Review of Last Week
- Exponential Families
- Generalized Linear Models

2 Generative Learning Algorithms
- Discriminative vs. Generative Learning Algorithms
- Gaussian Discriminant Analysis (GDA)
- Linear Discriminant Analysis (LDA)
- LDA and Logistic Regression
- MLE for GDA
- Naïve Bayes

Fit GDA: the training labels tell us which class to fit

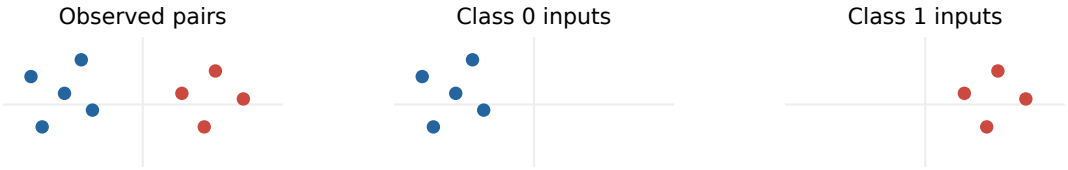
Known labels split the training points into classes; N_c counts the points in class c.



$$L(\theta) = \prod_n \pi_{y_n} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_{y_n}, \boldsymbol{\Sigma}_{y_n}).$$

Fit GDA: the training labels tell us which class to fit

Known labels split the training points into classes; N_c counts the points in class c.



$$L(\theta) = \prod_n \pi_{y_n} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_{y_n}, \boldsymbol{\Sigma}_{y_n}).$$

Taking logs groups the observations by class:

$$\log L = \underbrace{\sum_c N_c \log \pi_c}_{\text{class frequencies}} + \underbrace{\sum_c \sum_{n:y_n=c} \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)}_{\text{fit each class's inputs}}.$$

Fit GDA: count, find the center, measure the spread

Let $N_c = \sum_n \mathbf{1}\{y_n = c\} > 0$. For class-specific covariance (QDA):

1. Count

Class frequency.

$$\hat{\pi}_c = \frac{N_c}{N}.$$

2. Center

Average its inputs.

$$\hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{x}_n.$$

3. Spread

Center within this class:

$$\mathbf{r}_n = \mathbf{x}_n - \hat{\boldsymbol{\mu}}_c.$$

$$\hat{\boldsymbol{\Sigma}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{r}_n \mathbf{r}_n^\top.$$

Fit GDA: count, find the center, measure the spread

Let $N_c = \sum_n \mathbf{1}\{y_n = c\} > 0$. For class-specific covariance (QDA):

1. Count

Class frequency.

$$\hat{\pi}_c = \frac{N_c}{N}.$$

2. Center

Average its inputs.

$$\hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{x}_n.$$

3. Spread

Center within this class:

$$\mathbf{r}_n = \mathbf{x}_n - \hat{\boldsymbol{\mu}}_c.$$

$$\hat{\boldsymbol{\Sigma}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{r}_n \mathbf{r}_n^\top.$$

Again, match means within each class. The covariance MLE divides by N_c.

The full-dimensional density requires nonsingular scatter. Next, share one covariance.

MLE for LDA: write the shared-covariance objective

Use one covariance Σ for every class. With $\mathbf{r}_n = \mathbf{x}_n - \boldsymbol{\mu}_{y_n}$,

$$\log L(\theta) = \sum_c N_c \log \pi_c - \frac{N}{2} \log |\boldsymbol{\Sigma}| - \frac{1}{2} \sum_n \mathbf{r}_n^\top \boldsymbol{\Sigma}^{-1} \mathbf{r}_n - \frac{Nd}{2} \log(2\pi).$$

MLE for LDA: write the shared-covariance objective

Use one covariance Σ for every class. With $\mathbf{r}_n = \mathbf{x}_n - \boldsymbol{\mu}_{y_n}$,

$$\log L(\theta) = \sum_c N_c \log \pi_c - \frac{N}{2} \log |\boldsymbol{\Sigma}| - \frac{1}{2} \sum_n \mathbf{r}_n^\top \boldsymbol{\Sigma}^{-1} \mathbf{r}_n - \frac{Nd}{2} \log(2\pi).$$

- Priors: maximize $\sum_c N_c \log \pi_c$ subject to $\sum_c \pi_c = 1$.

MLE for LDA: write the shared-covariance objective

Use one covariance Σ for every class. With $\mathbf{r}_n = \mathbf{x}_n - \boldsymbol{\mu}_{y_n}$,

$$\log L(\theta) = \sum_c N_c \log \pi_c - \frac{N}{2} \log |\boldsymbol{\Sigma}| - \frac{1}{2} \sum_n \mathbf{r}_n^\top \boldsymbol{\Sigma}^{-1} \mathbf{r}_n - \frac{Nd}{2} \log(2\pi).$$

- Priors: maximize $\sum_c N_c \log \pi_c$ subject to $\sum_c \pi_c = 1$.
- Means: $\nabla_{\boldsymbol{\mu}_c}(-\log L) = \boldsymbol{\Sigma}^{-1} \sum_{n:y_n=c} (\boldsymbol{\mu}_c - \mathbf{x}_n) = 0$.

MLE for LDA: write the shared-covariance objective

Use one covariance Σ for every class. With $\mathbf{r}_n = \mathbf{x}_n - \boldsymbol{\mu}_{y_n}$,

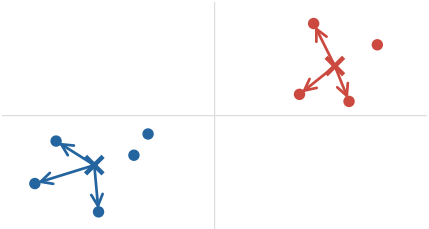
$$\log L(\theta) = \sum_c N_c \log \pi_c - \frac{N}{2} \log |\boldsymbol{\Sigma}| - \frac{1}{2} \sum_n \mathbf{r}_n^\top \boldsymbol{\Sigma}^{-1} \mathbf{r}_n - \frac{Nd}{2} \log(2\pi).$$

- Priors: maximize $\sum_c N_c \log \pi_c$ subject to $\sum_c \pi_c = 1$.
- Means: $\nabla_{\boldsymbol{\mu}_c}(-\log L) = \boldsymbol{\Sigma}^{-1} \sum_{n:y_n=c} (\boldsymbol{\mu}_c - \mathbf{x}_n) = 0$.
- The result is the within-class sample mean, just as in the shared-parameter MLE review.

Shared covariance: pool the spreads, not the centers

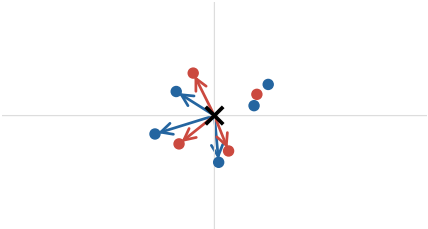
Subtract each point's own class mean before combining the residuals.

Points and their class centers



x_n

Move each class center to zero

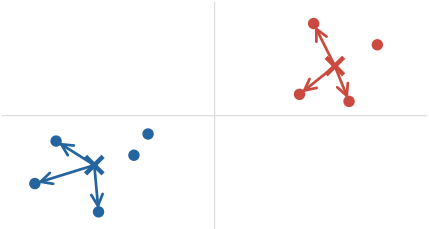


$r_n = x_n - \hat{\mu}_{y_n}$

Shared covariance: pool the spreads, not the centers

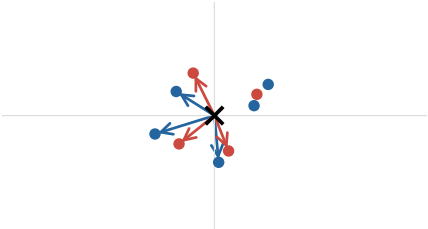
Subtract each point's own class mean before combining the residuals.

Points and their class centers



x_n

Move each class center to zero



$r_n = x_n - \hat{\mu}_{y_n}$

Each arrow keeps its direction and length when its class center moves to zero.

Then pool: estimate the shared covariance from all these residuals.

MLE for LDA: pool the within-class residuals

$$\hat{\pi}_c = \frac{N_c}{N}, \qquad \hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{x}_n.$$

MLE for LDA: pool the within-class residuals

$$\hat{\pi}_c = \frac{N_c}{N}, \qquad \hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{x}_n.$$

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{N} \sum_{n=1}^{N} (\mathbf{x}_n - \hat{\boldsymbol{\mu}}_{y_n})(\mathbf{x}_n - \hat{\boldsymbol{\mu}}_{y_n})^\top.$$

MLE for LDA: pool the within-class residuals

$$\hat{\pi}_c = \frac{N_c}{N}, \qquad \hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{x}_n.$$

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{N} \sum_{n=1}^{N} (\mathbf{x}_n - \hat{\boldsymbol{\mu}}_{y_n})(\mathbf{x}_n - \hat{\boldsymbol{\mu}}_{y_n})^\top.$$

- Subtract each point's **class mean**, not the overall mean.

MLE for LDA: pool the within-class residuals

$$\hat{\pi}_c = \frac{N_c}{N}, \qquad \hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{x}_n.$$

$$\boxed{\hat{\boldsymbol{\Sigma}} = \frac{1}{N} \sum_{n=1}^{N} (\mathbf{x}_n - \hat{\boldsymbol{\mu}}_{y_n})(\mathbf{x}_n - \hat{\boldsymbol{\mu}}_{y_n})^\top.}$$

- Subtract each point's **class mean**, not the overall mean.
- Average the within-class scatter over all N observations.

MLE for LDA: pool the within-class residuals

$$\hat{\pi}_c = \frac{N_c}{N}, \qquad \hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{x}_n.$$

$$\boxed{\hat{\boldsymbol{\Sigma}} = \frac{1}{N} \sum_{n=1}^{N} (\mathbf{x}_n - \hat{\boldsymbol{\mu}}_{y_n})(\mathbf{x}_n - \hat{\boldsymbol{\mu}}_{y_n})^\top.}$$

- Subtract each point's **class mean**, not the overall mean.
- Average the within-class scatter over all N observations.
- If singular, a practical modification is $\hat{\boldsymbol{\Sigma}}_\lambda = \hat{\boldsymbol{\Sigma}} + \lambda \mathbf{I}$, $\lambda > 0$; choose λ on validation data.

MLE for LDA: pool the within-class residuals

$$\hat{\pi}_c = \frac{N_c}{N}, \qquad \hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{x}_n.$$

$$\boxed{\hat{\boldsymbol{\Sigma}} = \frac{1}{N} \sum_{n=1}^{N} (\mathbf{x}_n - \hat{\boldsymbol{\mu}}_{y_n})(\mathbf{x}_n - \hat{\boldsymbol{\mu}}_{y_n})^\top.}$$

- Subtract each point's **class mean**, not the overall mean.
- Average the within-class scatter over all N observations.
- If singular, a practical modification is $\hat{\boldsymbol{\Sigma}}_\lambda = \hat{\boldsymbol{\Sigma}} + \lambda \mathbf{I}$, $\lambda > 0$; choose λ on validation data.
- To obtain $\mathbf{w}$, solve $\hat{\boldsymbol{\Sigma}}_\lambda \mathbf{w} = \hat{\boldsymbol{\mu}}_1 - \hat{\boldsymbol{\mu}}_0$ rather than forming the matrix inverse explicitly.

Table of Contents

1 Review of Last Week
- Exponential Families
- Generalized Linear Models

2 Generative Learning Algorithms
- Discriminative vs. Generative Learning Algorithms
- Gaussian Discriminant Analysis (GDA)
- Linear Discriminant Analysis (LDA)
- LDA and Logistic Regression
- MLE for GDA
- Naïve Bayes

From GDA to Naïve Bayes: the same recipe

Both learn the class prior and the inputs within each class, then apply Bayes' rule:

$$p(y = c \mid \mathbf{x}) \propto \underbrace{p(y = c)}_{\text{class prior}} \quad \underbrace{p(\mathbf{x} \mid y = c)}_{\text{class-conditional input model}} .$$

From GDA to Naïve Bayes: the same recipe

Both learn the class prior and the inputs within each class, then apply Bayes' rule:

$$p(y = c \mid \mathbf{x}) \propto \underbrace{p(y = c)}_{\text{class prior}} \quad \underbrace{p(\mathbf{x} \mid y = c)}_{\text{class-conditional input model}} .$$

What changes is the class-conditional model:

GDA: continuous features

$$p(\mathbf{x} \mid y = c) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c).$$

A multivariate Gaussian; covariance can capture feature correlations.

Here: binary word-presence features

$$p(\mathbf{x} \mid y = c) = \prod_j p(x_j \mid y = c).$$

One Bernoulli per word; assume features are independent given the class.

From GDA to Naïve Bayes: the same recipe

Both learn the class prior and the inputs within each class, then apply Bayes' rule:

$$p(y = c \mid \mathbf{x}) \propto \underbrace{p(y = c)}_{\text{class prior}} \quad \underbrace{p(\mathbf{x} \mid y = c)}_{\text{class-conditional input model}} .$$

What changes is the class-conditional model:

GDA: continuous features

$$p(\mathbf{x} \mid y = c) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c).$$

A multivariate Gaussian; covariance can capture feature correlations.

Here: binary word-presence features

$$p(\mathbf{x} \mid y = c) = \prod_j p(x_j \mid y = c).$$

One Bernoulli per word; assume features are independent given the class.

Same classification procedure; different distribution and feature assumptions.

Next: represent an email by which words are present.

Naïve Bayes: turn an email into binary features

Task: classify an email as spam ($y = 1$) or non-spam ($y = 0$).
Fix a dictionary of words. For each word j,

$$x_j = \begin{cases} 1, & \text{the word occurs in the email,} \\ 0, & \text{the word does not occur.} \end{cases}$$

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{matrix} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{matrix}$$

Naïve Bayes: turn an email into binary features

Task: classify an email as spam ($y = 1$) or non-spam ($y = 0$).
Fix a dictionary of words. For each word j,

$$x_j = \begin{cases} 1, & \text{the word occurs in the email,} \\ 0, & \text{the word does not occur.} \end{cases}$$

For dictionary `[offer, meeting]`,

"`offer offer`" becomes $\mathbf{x} = (1, 0)$.

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{matrix} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{matrix}$$

Naïve Bayes: turn an email into binary features

Task: classify an email as spam ($y = 1$) or non-spam ($y = 0$).
Fix a dictionary of words. For each word j,

$$x_j = \begin{cases} 1, & \text{the word occurs in the email,} \\ 0, & \text{the word does not occur.} \end{cases}$$

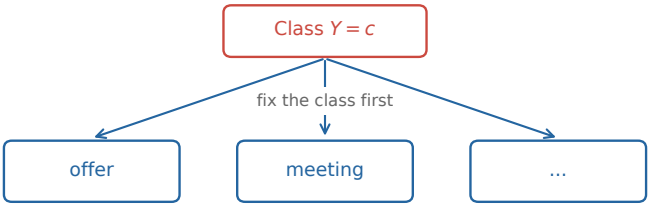
For dictionary `[offer, meeting]`,

"`offer offer`" becomes $\mathbf{x} = (1, 0)$.

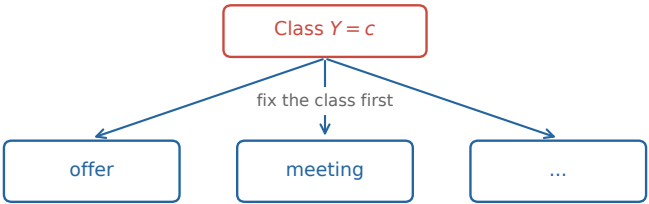
We record **presence**, not the number of occurrences.

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{matrix} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{matrix}$$

What is “naïve”? Fix the class, then multiply



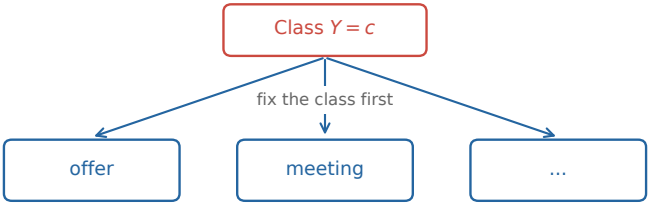
What is “naïve”? Fix the class, then multiply



Assumption: word-presence features are independent **given the class**.

$$p(\mathbf{x} \mid y = c) = \prod_{j=1}^{d} p(x_j \mid y = c).$$

What is “naïve”? Fix the class, then multiply



Assumption: word-presence features are independent **given the class**.

$$p(\mathbf{x} \mid y = c) = \prod_{j=1}^{d} p(x_j \mid y = c).$$

For example, within spam, model `offer` and `meeting` separately.

Real words can still be related; this is a simplifying model assumption.

What probabilities do we need?

For each candidate class c, learn:

- **Class prior:** $\pi_c = p(y = c)$, the fraction of emails in this class.

What probabilities do we need?

For each candidate class c, learn:

- **Class prior:** $\pi_c = p(y = c)$, the fraction of emails in this class.
- **Word probability:** $q_{jc} = p(x_j = 1 \mid y = c)$,
  how often word j occurs in emails of this class.

What probabilities do we need?

For each candidate class c, learn:

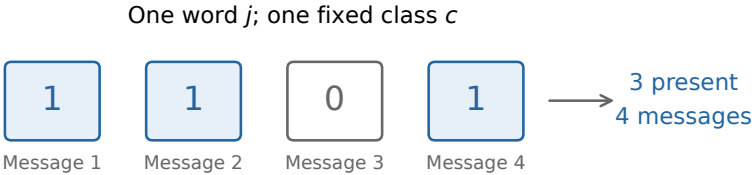
- **Class prior:** $\pi_c = p(y = c)$, the fraction of emails in this class.
- **Word probability:** $q_{jc} = p(x_j = 1 \mid y = c)$,
  how often word j occurs in emails of this class.

For a new email, each word selects one factor:

$$p(x_j \mid y = c) = \begin{cases} q_{jc}, & x_j = 1 \quad \text{(present)}, \\ 1 - q_{jc}, & x_j = 0 \quad \text{(absent)}. \end{cases}$$

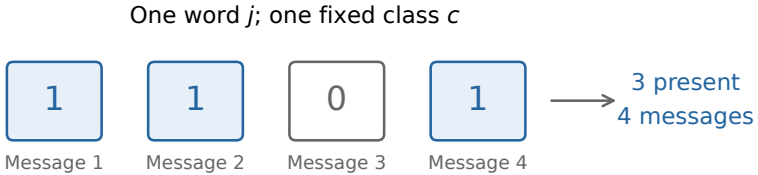
Training: count within each class

Suppose word j occurs in three of four emails in class c.



Training: count within each class

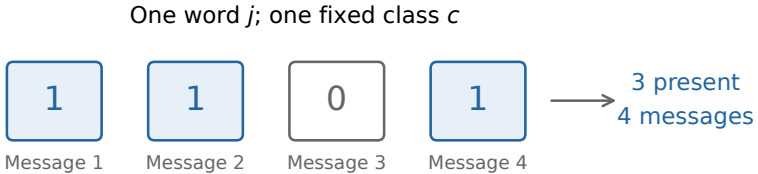
Suppose word j occurs in three of four emails in class c.



$$\hat{q}_{jc} = \frac{\text{class-}c\text{ emails containing word } j}{\text{all class-}c\text{ emails}} = \frac{N_{jc}}{N_c} = \frac{3}{4}.$$

Training: count within each class

Suppose word j occurs in three of four emails in class c.



$$\hat{q}_{jc} = \frac{\text{class-}c\text{ emails containing word } j}{\text{all class-}c\text{ emails}} = \frac{N_{jc}}{N_c} = \frac{3}{4}.$$

Repeat for every word and class. Estimate the prior by $\hat{\pi}_c = N_c/N$.

Training is just counting; no iterative optimizer is needed.

Prediction: multiply, then compare

Use the learned probabilities to compute one weight for each class:

$$w_c = \underbrace{\pi_c}_{\text{class prior}} \prod_{j=1}^{d} \underbrace{p(x_j \mid y = c)}_{\text{present: } q_{jc} \text{ ; absent: } 1-q_{jc}} .$$

Prediction: multiply, then compare

Use the learned probabilities to compute one weight for each class:

$$w_c = \underbrace{\pi_c}_{\text{class prior}} \prod_{j=1}^{d} \underbrace{p(x_j \mid y = c)}_{\text{present: } q_{jc} \text{ ; absent: } 1-q_{jc}} .$$

Choose the class with the larger weight. Both present and absent words contribute.

Prediction: multiply, then compare

Use the learned probabilities to compute one weight for each class:

$$w_c = \underbrace{\pi_c}_{\text{class prior}} \prod_{j=1}^{d} \underbrace{p(x_j \mid y = c)}_{\text{present: } q_{jc} \text{ ; absent: } 1-q_{jc}} .$$

Choose the class with the larger weight. Both present and absent words contribute.

If we also want a probability, normalize the weights:

$$p(y = 1 \mid \mathbf{x}) = \frac{w_1}{w_0 + w_1}.$$

The weights already include the class priors: this is Bayes' rule.

One email, two words: work it out

Dictionary: `[offer, meeting]`. The email has `offer`, but no `meeting`: $\mathbf{x} = (1, 0)$.

	Prior	`offer`	no `meeting`	Weight

One email, two words: work it out

Dictionary: `[offer, meeting]`. The email has `offer`, but no `meeting`: $\mathbf{x} = (1, 0)$.

	Prior		`offer`		no `meeting`		Weight
Spam	0.5	$\times$	0.8	$\times$	$1 - 0.2$	$=$	0.32
Non-spam	0.5	$\times$	0.1	$\times$	$1 - 0.7$	$=$	0.015

One email, two words: work it out

Dictionary: `[offer, meeting]`. The email has `offer`, but no `meeting`: $\mathbf{x} = (1, 0)$.

	Prior		`offer`		no `meeting`		Weight
Spam	0.5	$\times$	0.8	$\times$	$1 - 0.2$	$=$	0.32
Non-spam	0.5	$\times$	0.1	$\times$	$1 - 0.7$	$=$	0.015

$$p(\text{spam} \mid \mathbf{x}) = \frac{0.32}{0.32 + 0.015} \approx 0.955.$$

Predict **spam**. For the absent word, use $1 - q$, not q.

Laplace smoothing: an unseen event need not be impossible

If a word never appeared in class c, its estimated presence probability is 0.

A new email containing it would make that class's entire weight 0.



Laplace smoothing: an unseen event need not be impossible

If a word never appeared in class c, its estimated presence probability is 0.

A new email containing it would make that class's entire weight 0.



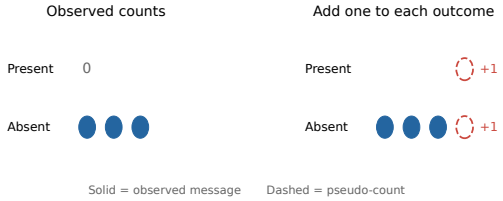
Add one pseudo-count to **presence** and one to **absence**:

$$\tilde{q}_{jc} = \frac{N_{jc} + 1}{N_c + 2}.$$

Laplace smoothing: an unseen event need not be impossible

If a word never appeared in class c, its estimated presence probability is 0.

A new email containing it would make that class's entire weight 0.



Add one pseudo-count to **presence** and one to **absence**:

$$\tilde{q}_{jc} = \frac{N_{jc} + 1}{N_c + 2}.$$

With zero occurrences in three emails: $0/3 \longrightarrow (0+1)/(3+2) = 1/5$.

The denominator gains 2 because a binary feature has two possible outcomes.

Naïve Bayes: the basic idea

- **Assume:** features are independent given the class.
- **Train:** count class frequencies and word presence within each class; add pseudo-counts to avoid zero word probabilities.

Naïve Bayes: the basic idea

- **Assume:** features are independent given the class.
- **Train:** count class frequencies and word presence within each class; add pseudo-counts to avoid zero word probabilities.
- **Predict:** multiply the prior by the word factors for each class, then choose the larger weight. Normalize if a probability is needed.

The same generative recipe as GDA: model $p(\mathbf{x} \mid y)$ and $p(y)$, then use Bayes' rule.